Institut Supérieur d'Informatique
Département génie télécommunication et réseaux (GTR)

Module : Cloud Computing & Virtualisation

Groupes : M1 SSII

Enseignant : Safa Réjichi

Nombre de pages : 3

@mail : safa.rejichi@isi.utm.tn

Travaux Pratiques 3 : Orchestration avec Docker compose

Objectifs :

- ⇨ Manipulation de docker compose
- ⇨ Orchestration de deux services Application Web et Database avec Docker compose

Partie 1 : Installation de docker-compose

Docker Compose est utilisé pour contrôler plusieurs conteneurs sur un même système. Tout comme le Dockerfile. Il y a un fichier texte qui décrit l'application : quelles images utiliser, combien d'instances, les connexions réseau, etc.

I. Installez docker-compose.

Docker Compose n'est pas installé par défaut et s'appuie sur le moteur Docker pour fonctionner. Voici la procédure à suivre pour télécharger Docker Compose sous un environnement Linux :

```
sudo apt-get update
sudo apt-get install docker-compose-plugin
```

II. Vérifiez ensuite votre installation :

```
docker compose –version
```

Partie 2 : Orchestration avec docker-compose

I. Créez et démarrez vos conteneurs à partir du fichier docker-compose.yml. Pour faire les choses dans les règles de l'art, nous allons séparer la base et l'application web chacun dans un conteneur. Ceci sera fait comme suit :

1. Créez un nouveau Dockerfile où nous allons enlever la couche mysql comme suit :

```
# --------------- DÉBUT COUCHE OS -------------------
FROM ubuntu:latest
# --------------- FIN COUCHE OS ---------------------
# MÉTADONNÉES DE L'IMAGE
LABEL version="1.0" maintainer="NOM Prénom <@votre_adresse_mail>"
# VARIABLES TEMPORAIRES
ARG APT_FLAGS="-q -y"
ARG DOCUMENTROOT="/var/www/html"
# --------------- DÉBUT COUCHE APACHE ---------------
RUN apt-get update -y && DEBIAN_FRONTEND=noninteractive apt-get install
${APT_FLAGS} apache2
# --------------- FIN COUCHE APACHE -----------------
# --------------- DÉBUT COUCHE PHP ------------------
RUN DEBIAN_FRONTEND=noninteractive apt-get install ${APT_FLAGS} \
    php-mysql \
    php && \
    rm -f ${DOCUMENTROOT}/index.html && \
    apt-get autoclean -y
COPY sources/app ${DOCUMENTROOT}
# --------------- FIN COUCHE PHP --------------------
# OUVERTURE DU PORT HTTP
EXPOSE 80
# RÉPERTOIRE DE TRAVAIL
WORKDIR  ${DOCUMENTROOT}
# DÉMARRAGE DES SERVICES LORS DE L'EXÉCUTION DE L'IMAGE

ENTRYPOINT apache2ctl -D FOREGROUND
```

1. Ensuite, créez un fichier dans le même répertoire et nommez le docker-compose.yml. Rajoutez le contenu suivant :

```
services:
    db:
        image: mysql:9.5.0
        container_name: mysql_c
        restart: always
        volumes:
            - db-volume:/var/lib/mysql
            - ./sources/db/articles.sql:/docker-entrypoint-initdb.d/articles.sql
        environment:
            MYSQL_ALLOW_EMPTY_PASSWORD: "yes"
            MYSQL_DATABASE: test
            MYSQL_USER: test
            MYSQL_PASSWORD: test
        ports:
            - 3306:3306
        networks:
            - app_net
        command: >
          bash -c "
          docker-entrypoint.sh mysqld &
          pid=$!;
          echo 'Waiting for MySQL...';
          until mysql -u root --password= -e 'SELECT 1;' > /dev/null 2>&1; do
            sleep 2;
          done;
          echo 'Running init script...';
          mysql -u root --password= < /docker-entrypoint-initdb.d/articles.sql;
          wait $pid
          "
        healthcheck:
            test: ["CMD", "mysql", "--user=test", "--password=test", "--execute", "SELECT 1;"]
            interval: 10s
            timeout: 5s
            retries: 5
            start_period: 30s
```

```
    app:
        build:
            context: .
            dockerfile: Dockerfile
        image: my_lamp_compose
        restart: always
        ports:
            - 8081:80
        container_name: my_lamp_compose_c
        depends_on:
            db:
              condition: service_healthy
        networks:
            - app_net
volumes:
    db-volume:

networks:
  app_net:
    driver: bridge
```

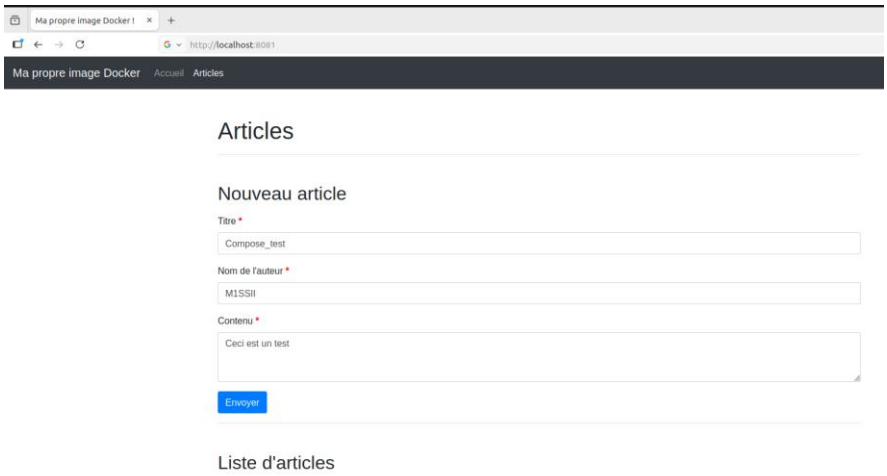
Voici à quoi doit ressembler votre arborescence :

```
├──sources
│      ├──app
│      │    ├── db-config.php
│      │    ├── index.php
│      │    └── validation.php
│      └── db
│           └── articles.sql
├── docker-compose.yml
└── Dockerfile
```

2. Expliquez le contenu du fichier docker-compose.yml ?
3. Dans le fichier de configuration « db-config.php » au niveau de notre application web, changez l'hôte de « localhost » au nouveau conteneur « db » crée contenant notre base de données ? Expliquez ?
4. Créez vos conteneurs en utilisant le fichier docker-compose.yml avec la commande :
   docker compose up -d
5. En cas de besoin, donnez les privilèges aux fichiers nécessaires au fonctionnement de votre application web avec :
   docker exec my_lamp_compose_c chmod 777 -R /var/www/html

II. Visitez ensuite la page http://localhost:8081/ et affichez le résultat.

III. Ajoutez un article comme suit et vérifiez que l'article a été bien ajouté :



IV. Donnez les adresses ip de vos conteneurs en utilisant la commande docker inspect? Expliquez leur relation avec app_net ?